

Degraded-Gimbal Case Study

Apollo LM Powered Descent with One Sluggish TVC Actuator

30 July 2026

Contents

1	Degraded-Gimbal Case Study	1
1.1	The headline result	2
1.2	Results	2
1.2.1	Tracking error grows roughly six-fold	3
1.2.2	The compensation is counterintuitive	3
1.2.3	The degradation does not change which constraints bind	3
1.3	Repeatability, and what the objective can and cannot tell you	3
1.4	The mechanism, and an important caveat	4
1.5	What the model does and does not capture	5

1 Degraded-Gimbal Case Study

Engine 2’s gimbal actuator is made progressively more sluggish while engine 1 stays healthy. Vehicle, scenario, grid, and cost weights are held at the baseline, so every difference between runs comes from that one actuator.

The gimbal is a second-order servo per axis,

$$\ddot{\delta} = \omega_n^2 (\delta_c - \delta) - 2\zeta\omega_n \dot{\delta},$$

so a degradation is a change to (ω_n, ζ) for one engine, applied through the new `LMPParams.gimbal_wn_eng` / `gimbal_zeta_eng` overrides.

Case	Directory	ω_n	ζ	−3dB BW	Settling	Overshoot
G1	G1_nominal/	4.0 rad/s	0.70	4.04 rad/s	1.4 s	5 %
G2	G2_mild_wn1.5/	1.5 rad/s	0.70	1.52 rad/s	3.8 s	5 %
G3	G3_severe_wn0.6/	0.6 rad/s	0.70	0.61 rad/s	9.5 s	5 %
G4	G4_underdamped_wn1.5_zeta0.25/	1.5 rad/s	0.25	1.48 rad/s	16.0 s	44 %

G2 and G3 model a loss of actuator bandwidth — degraded hydraulic supply pressure or a worn servo valve — which makes the gimbal *lag*. G4 models a loss of damping as well, so the gimbal *rings*

instead of merely lagging. Only the gimbal changes; the degraded engine’s thrust and its first-order thrust lag are untouched.

Each directory holds `states.png`, `controls.png`, `actuators.png`, `thrusters.png`, `trajectory_with_axes.png`, `console_log.txt`, `metrics.json`, and `solution.npz`. Three cross-case figures sit at the top level:

- `actuator_step_response.png` — the four actuator models on their own, before any trajectory is involved
- `gimbal_tracking.png` — small multiples of engine 2’s yaw gimbal, commanded versus actual. **This is the figure that carries the result.**
- `degradation_metrics.png` — what the degradation costs

Reproduce with:

```
python run_gimbal_study.py          # all four cases, about 8 min
python plot_actuator_response.py    # analytic actuator comparison (no solve)
python plot_gimbal_comparison.py    # cross-case figures + summary table
```

The driver pins BLAS/OpenMP to one thread before importing numpy — see §1.3, this is necessary for the numbers to be reproducible.

1.1 The headline result

All four cases land, to the same precision. Degrading one gimbal is a graceful performance loss — not a cliff.

This is worth stating alongside the engine-out study, which found the opposite: losing an engine at the design spacing makes the problem *infeasible*, with no solution at any cost. A sluggish gimbal is a completely different kind of fault. The vehicle has redundant roll/pitch authority — a second gimbal, differential throttling, and 16 RCS thrusters — so the optimiser routes around a slow actuator. It cannot route around a moment it has no authority to cancel.

Every case reaches the contact altitude with terminal speed below 10^{-5} m/s, zero horizontal error, and attitude error below 10^{-4} °.

1.2 Results

Metric	G1	G2	G3	G4
Objective J	2.867e8	3.507e8	3.196e8	3.406e8
J vs baseline	—	+22.3 %	+11.5 %	+18.8 %
E2 gimbal RMS tracking error	0.477°	2.888°	2.699°	3.517°
E2 gimbal <i>max</i> tracking error	2.365°	12.000°	9.925°	12.000°
E2 RMS gimbal command	1.510°	2.065°	2.300°	2.697°
E1 RMS gimbal command	1.600°	1.328°	1.420°	1.547°

Metric	G1	G2	G3	G4
Peak differential thrust	5 659 N	4 263 N	5 905 N	5 260 N
RCS impulse	91 068 N · s	97 544 N · s	92 218 N · s	98 510 N · s
DPS impulse	1.716e6 N · s	1.681e6 N · s	1.682e6 N · s	1.689e6 N · s
Peak $ \phi $	45.00°	45.00°	45.00°	45.00°
Peak $ p $	10.00 °/s	10.00 °/s	10.00 °/s	10.00 °/s
Terminal speed	3.8e-6 m/s	1.8e-6 m/s	8.5e-6 m/s	1.4e-6 m/s
IPOPT iterations	168	390	270	284

1.2.1 Tracking error grows roughly six-fold

The degraded gimbal’s RMS command-tracking error rises from 0.48° at baseline to 2.7–3.5° — a factor of 5.7 to 7.4. The *maximum* error reaches exactly 12.000° in G2 and G4, which is full scale: the command sits at one $\pm 6^\circ$ limit while the actual deflection sits at the other. The actuator is, at moments, doing the precise opposite of what it was told.

`gimbal_tracking.png` shows this directly. G1’s actual deflection follows its command closely. G3’s command swings between the $\pm 6^\circ$ limits while the actual deflection never exceeds $\pm 3^\circ$ — heavy attenuation. G4’s response rings, crossing its command repeatedly.

1.2.2 The compensation is counterintuitive

The natural expectation is that the optimiser abandons the bad actuator and leans on the healthy one. **It does the opposite.**

The RMS *command* sent to the degraded gimbal rises monotonically with severity — $1.51^\circ \rightarrow 2.07^\circ \rightarrow 2.30^\circ \rightarrow 2.70^\circ$ — while the healthy engine’s command **falls**, from 1.60° to 1.33–1.55°. RCS use moves by only +1.3 % to +8.2 %, and differential thrust shows no trend (4 263–5 905 N, unordered).

The reason is that a sluggish actuator **attenuates**: to obtain a given deflection you must over-drive it. The optimiser therefore commands harder, not elsewhere. The degraded engine keeps contributing roll and pitch authority, just less of it per unit of command, and the optimiser pays for that in control effort rather than by substituting a different actuator.

This trend is the most robust quantitative finding in the study — it holds in the same direction and magnitude across two independent solve campaigns (see below), unlike the objective.

1.2.3 The degradation does not change which constraints bind

Peak roll is 45.00° and peak roll rate 10.00 °/s in *every* case, baseline included — the attitude and rate limits are saturated regardless. The degradation does not change the active constraint set; it changes how expensive it is to respect it.

1.3 Repeatability, and what the objective can and cannot tell you

The objective differences between cases are not reliable, and the ordering $G2 > G4 > G3$ in the table above should not be interpreted as a severity ranking. This is a property of the method, not of the hardware, and it is worth documenting carefully.

Two things were measured:

1. Multithreaded BLAS makes the solve nondeterministic. Three runs of the *identical* nominal NLP with default threading gave $J = 2.8675\text{e}8$, $2.8670\text{e}8$ and $2.8391\text{e}8$ — a 1.0 % spread. The problem is nonconvex, so a change in floating-point reduction order is enough to steer IPOPT into a different local minimum. With `OMP_NUM_THREADS=1` (and the OpenBLAS/MKL equivalents), three runs gave $2.867018\text{e}8$ to every printed digit, and 168 iterations each. The driver now pins these before importing numpy, so the tabulated numbers are reproducible.

2. Pinning threads does not make the cases *comparable*. Comparing the two independent campaigns for each case:

Case	Campaign 1 (unpinned)	Campaign 2 (pinned)	Difference
G1	2.8391e8	2.8670e8	1.0 %
G2	3.1755e8	3.5069e8	10.4 %
G3	3.1956e8	3.1960e8	0.01 %
G4	3.4181e8	3.4058e8	0.4 %

G2’s objective moved by 10.4 % between campaigns — comparable to the entire spread *between* cases. Each solve is now individually reproducible, but the particular local minimum a given (ω_n, ζ) leads to is not a smooth function of the degradation, so the between-case objective gap carries a several-percent uncertainty that no amount of thread pinning removes.

What survives this, and what does not:

- **Robust:** all cases land; tracking error grows several-fold; the optimiser over-commands the degraded gimbal and *reduces* the healthy engine’s command; RCS use changes by only single-digit percent; the active constraint set is unchanged. All of these held in both campaigns.
- **Not robust:** the ordering of J between cases. The honest statement is that degrading one gimbal costs **on the order of 10–20 % in objective**, without resolving which of these three degradations is worst.

In the first campaign the ordering happened to be monotone in severity (+11.9 %, +12.6 %, +20.4 %), which invited the tidy conclusion that losing damping costs more than losing bandwidth. The second campaign (+22.3 %, +11.5 %, +18.8 %) does not support it. That conclusion has been withdrawn rather than kept because it read well.

To resolve the ranking properly, each case would need to be solved from a spread of initial guesses (a multi-start) and the best minimum found for each retained — which is the standard remedy for exactly this problem and would be the natural next step.

1.4 The mechanism, and an important caveat

The right panel of `actuator_step_response.png` explains why over-commanding works. The command grid updates once per second, so its Nyquist rate is $\pi/\Delta t = 3.14$ rad/s. The healthy actuator’s –3 dB bandwidth is 4.04 rad/s — **above** that line, so it can follow anything the optimiser is allowed to command. All three degraded actuators sit below it (1.52, 0.61, 1.48 rad/s), so they act as low-pass filters on the command sequence.

The optimiser discovers this and exploits it: it issues a fast, large-amplitude command whose *filtered* response is the deflection profile it actually wants. That is physically real — a real actuator does filter its command — but two caveats apply to reading these numbers as hardware predictions:

1. **The commands are not realistic GNC outputs.** No flight autopilot commands $\pm 6^\circ$ square waves at 1 Hz. The optimiser is free to because the cost penalises the *commanded* rate (R_d on Δu) rather than the achieved deflection rate, and R_d was tuned for the healthy actuator. Penalising $\dot{\delta}$ directly as a state, or re-tuning R_d per engine, would suppress the chatter and probably raise the reported cost of degradation.
2. **The result is somewhat grid-dependent.** A finer grid would let the optimiser chatter faster and be filtered harder. Note this cuts the opposite way from the baseline model’s usual concern: $\Delta t = 1$ s under-resolves the *healthy* actuator ($\omega_n = 4$ rad/s) but comfortably resolves the sluggish ones, so the degraded cases are the better-resolved of the four.

A useful follow-up is to repeat G3 with $\Delta t = 0.5$ s and a per-engine R_d , and check whether the penalty holds.

1.5 What the model does and does not capture

The degradation is **present from $t = 0$ and constant** for the whole horizon, and the optimiser plans with full knowledge of it. These runs are therefore a study of *flying a known-degraded vehicle*, not of *responding to a degradation*.

Not modelled:

- **Onset partway through the descent**, which would be a transient recovery problem from a worse state, with a detection delay the open-loop OCP cannot represent.
- **Rate and acceleration saturation** in the actuator. The servo here is linear with no slew-rate limit, so it attenuates gracefully. A real degraded actuator is likely to hit a hard rate limit — a nonlinearity that would hurt more than the loss of bandwidth modelled here, and one that would also block the over-commanding strategy the optimiser relies on.
- **Asymmetric degradation between the two gimbal axes.** Both the pitch and yaw axes of engine 2 are degraded identically; a single stuck or slow axis is a different and probably harder fault.
- **Correlation with other faults.** A failing actuator often signals a failing hydraulic or electrical supply that would affect thrust as well.
- **Feedback.** As throughout, these are open-loop reference trajectories.

Finally, the graceful behaviour here depends on there being a healthy second gimbal to work with. Degrading *both* gimbals, or combining a sluggish gimbal with the engine-out condition, would remove the redundancy this study is implicitly measuring — and the engine-out study shows how abruptly this vehicle fails once its roll authority is gone.